

DECISION LOG

CODEVERITAS

Trustworthy verification of practical software engineering skills

TEAM Code Breakers (TEAM-CB001)

DATE 27 September 2026

Q1. What approach did we take, and what did we reject?

Our approach: evidence-linked developer credentials. A developer submits a public repository for a role-specific challenge. We collect repository activity and, when the developer opts in, a recorded work session; the platform then uses that evidence for a repository-grounded PRI quiz and speaking assessment. Scores and source evidence stay attached to the challenge credential. Gemini can summarize submitted evidence, but it does not establish authorship or make the hiring decision. Two different reviewers score the submission with the same rubric; only after both reviews does the credential become verified and appear in CodePassport. This gives a recruiter a trail to inspect instead of an unexplained rank. **We rejected** a leaderboard based only on commit counts or one automated code-quality score. Both looked attractive because activity is cheap to collect and automated scores can be produced quickly. Neither shows who did the work or why a design fits its challenge, and a single model error could unfairly become a hiring signal.

Q2. Why did we reject it? What was the trade-off?

We compared the approaches on four dimensions. **Attribution:** repository plus session context and two reviews is stronger than commit volume, but still cannot prove authorship; both approaches have that limit. **Abuse resistance:** independent reviewers raise the cost of gaming, while commit counts are easy to inflate; reviewer collusion remains possible. **Connectivity:** our flow needs GitHub and Gemini access, so outages delay analysis; a local-only counter could work offline but would offer weaker evidence. **Delivery in 72 hours:** the shared challenge, assessment, and review flow took more work than a leaderboard, but gives reviewers inspectable records. We chose attribution over speed. The cost we accept is slower verification: every verified credential needs two reviewers, so a queue can grow before we can add reviewer assignment and workload controls. We also accept provider latency, API quotas, and the privacy burden of handling recordings. We limit collection to opted-in challenge evidence and keep recordings out of public CodePassports.

Q3. What breaks at the scale of all Mysuru?

CodeVeritas is a developer-assessment platform, not a civic complaint router; we apply the 65-ward, town, and panchayat scenario to a citywide hiring pilot. We model 3,000 submissions on a normal day and a 10x Dasara burst: 30,000 submissions/day, or about 21 per minute averaged across 24 hours. Using roughly 1.5 MB as a low observed recording size, that is about 45 GB/day of recording data at peak before retries or retention. The first failure would be local disk and upload handling: development recordings are stored on the application machine, with no durable object store or queue. SQLite writes and synchronous GitHub/Gemini calls would then add contention and quota delays. **Our first change** would be private object storage with resumable uploads, size limits, and automatic retention/deletion. Next we would move to a production database and queue provider work with bounded concurrency and retry backoff. We have not load-tested these estimates; they are planning assumptions, not measured capacity.

Planning estimates are explicit assumptions, not measured load-test results. | CodeVeritas · Code Breakers